Báo Cáo

Contents

Báo Cáo....1
1. Kiến trúc tổng thể và giao thức truyền thống....2
1.1 Kiến trúc tổng thể....2
1.2 Giao thức truyền thông và định dạng dữ liệu....3
2. Mô hình sao lưu và đảm bảo tính nhất quán được sử dụng. 5
2.1 Mô hình chiến lược và sao lưu....5
2.2 Mô hình nhất quán: Tính nhất quán sau cùng....6
3. Chiến lược xử lí node hỏng và khôi phục dữ liệu....6
3.1 Phát hiện lỗi và giao thức heartbeat....7
3.2. Tính chịu lỗi (fault tolerance) để đảm bảo tính sẵn sàng 7
3.3 Quy trình khôi phục dữ liệu (Node recovery)....8
4. Phương án phân chia và mở rộng....8
4.1 Phân chia dữ liệu: hash Partitioning....9
4.2 Chiến lược mở rộng và đề xuất cải tiến....9
5. Những hạn chế và đề xuất cải tiến....10
5.1. Hạn chế về mở rộng: chi phí tái phân bố dữ liệu cao....11
5.2. Hạn chế về độ bền dữ liệu (Durability)....11
5.3. Hạn chế về tính sẵn sàng của thao tác ghi....12
5.4. Hạn chế trong quy trình khôi phục....12

1. Kiến trúc tổng thể và giao thức truyền thống

Phần này mô tả kiến trúc ở mức cao của hệ thống lưu trữ key-value phân tán, cùng với các quyết định thiết kế liên quan đến giao thức giao tiếp và định dạng dữ liệu được sử dụng.

1.1 Kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình **cụm các node (cluster of nodes)** hoạt động theo kiểu **peer-to-peer (P2P)**. Không có một node chủ (master node) trung tâm nào điều phối toàn bộ hoạt động. Thay vào đó, mọi node trong hệ thống đều bình đẳng, có khả năng tự xử lý yêu cầu từ client và giao tiếp trực tiếp với các node khác.

Các thành phần chính:

1. **Node:**
    - Là đơn vị tính toán và lưu trữ cơ bản của hệ thống.
    - Mỗi node chịu trách nhiệm lưu trữ một phần dữ liệu của toàn bộ hệ thống.
    - Có khả năng tiếp nhận yêu cầu từ client (GET, PUT, DELETE).
    - Tự xác định node nào chịu trách nhiệm cho một key cụ thể và chuyển tiếp (forward) yêu cầu nếu cần.
    - Thực hiện sao lưu dữ liệu (replication) đến các node khác để đảm bảo tính sẵn sàng và chịu lỗi.
    - Giao tiếp với các node khác để theo dõi trạng thái (heartbeat) và phối hợp trong quá trình khôi phục dữ liệu.
2. **Client:**
    - Là một thực thể bên ngoài (ví dụ: một ứng dụng, một người dùng) muốn thực hiện các thao tác trên dữ liệu.
    - Client có thể kết nối đến **bất kỳ node nào** trong cụm để gửi yêu cầu. Điều này giúp đơn giản hóa logic phía client và tăng tính sẵn sàng của hệ thống.

Luồng hoạt động của một yêu cầu PUT:

Kiến trúc này được minh họa rõ nhất qua luồng xử lý một yêu cầu ghi dữ liệu (PUT):

1. **Client gửi yêu cầu:** Client tạo một yêu cầu PUT chứa key và value rồi gửi đến một node bất kỳ trong cụm (tạm gọi là Node A).

2. **Xác định Node sở hữu:** Node A nhận yêu cầu và sử dụng thuật toán **Hash Partitioning** (sẽ được mô tả chi tiết ở phần sau) để xác định node_id của node thực sự sở hữu key này (tạm gọi là Node B).
3. **Chuyển tiếp yêu cầu (nếu cần):**
    - Nếu Node A chính là Node B (tức Node A là chủ sở hữu của key), nó sẽ trực tiếp xử lý yêu cầu.
    - Nếu Node A không phải là Node B, nó sẽ chuyển tiếp toàn bộ yêu cầu PUT đến Node B.
4. **Xử lý và Sao lưu:** Node B (node sở hữu) nhận yêu cầu, ghi cặp key-value vào bộ lưu trữ cục bộ của mình. Ngay sau đó, nó sẽ gửi các yêu cầu sao lưu (REPLICA_PUT) đến các node bản sao (replicas) đã được cấu hình trước.
5. **Phản hồi cho Client:** Sau khi ghi dữ liệu cục bộ, Node B gửi một thông báo thành công về cho node đã chuyển tiếp yêu cầu (Node A). Cuối cùng, Node A gửi phản hồi thành công này lại cho client. Quá trình sao lưu diễn ra bất đồng bộ và không làm chậm phản hồi gửi về cho client.

1.2 Giao thức truyền thông và định dạng dữ liệu

Việc lựa chọn giao thức và định dạng dữ liệu là nền tảng cho việc giao tiếp hiệu quả và đáng tin cậy trong một hệ thống phân tán.

1.2.1. Giao thức truyền thông: TCP

Hệ thống sử dụng **TCP (Transmission Control Protocol)** cho toàn bộ việc giao tiếp, bao gồm giữa client và node, cũng như giữa các node với nhau. Lựa chọn này dựa trên các đặc tính quan trọng sau của TCP:

- **Độ tin cậy (Reliability):** TCP đảm bảo rằng mọi gói tin được gửi đi sẽ đến được đích. Nó sử dụng cơ chế xác nhận (acknowledgment), truyền lại (retransmission) các gói tin bị mất và kiểm tra tổng (checksum) để phát hiện lỗi dữ liệu. Đối với một hệ thống lưu trữ, việc đảm bảo yêu cầu (ví dụ: PUT) không bị mất hoặc hỏng trên đường truyền là cực kỳ quan trọng.
- **Thứ tự gói tin (Ordered Delivery):** TCP đảm bảo các gói tin được giao đến bên nhận theo đúng thứ tự mà chúng được gửi đi. Điều này giúp đơn giản hóa logic xử lý ở phía nhận, vì không cần phải tự sắp xếp lại các thông điệp bị xáo trộn.
- **Kiểm soát luồng (Flow Control):** TCP tự động điều chỉnh tốc độ truyền để tránh làm quá tải bên nhận, giúp hệ thống hoạt động ổn định dưới tải trọng cao.

So với UDP, mặc dù TCP có độ trễ cao hơn một chút do cơ chế bắt tay và xác nhận, nhưng sự đảm bảo về độ tin cậy và thứ tự mà nó mang lại là sự đánh đổi xứng đáng cho một hệ thống cơ sở dữ liệu phân tán.

1.2.2. Định dạng tuần tự hóa dữ liệu: JSON

Tất cả các thông điệp (yêu cầu và phản hồi) trong hệ thống đều được **tuần tự hóa (serialize)** sang định dạng **JSON (JavaScript Object Notation)** trước khi gửi qua mạng TCP.

- **Dễ đọc và gỡ lỗi (Human-Readable & Debuggable):** JSON là một định dạng văn bản, giúp cho việc theo dõi, phân tích và gỡ lỗi các thông điệp trên mạng trở nên đơn giản hơn rất nhiều so với các định dạng nhị phân.
- **Khả năng tương tác (Interoperability):** JSON được hỗ trợ rộng rãi bởi hầu hết các ngôn ngữ lập trình hiện đại, bao gồm cả Python. Điều này giúp hệ thống dễ dàng mở rộng hoặc tích hợp với các dịch vụ khác trong tương lai.
- **Linh hoạt (Flexible):** Cấu trúc key-value của JSON cho phép dễ dàng thêm các trường mới vào thông điệp mà không làm hỏng khả năng tương thích ngược của các node cũ hơn.

2. Mô hình sao lưu và đảm bảo tính nhất quán được sử dụng.

Phần này đi sâu vào chiến lược sao lưu dữ liệu trên nhiều node và mô hình nhất quán mà hệ thống cam kết, vốn là hệ quả trực tiếp từ các lựa chọn thiết kế này.

2.1 Mô hình chiến lược và sao lưu

Để đảm bảo tính sẵn sàng cao (high availability) và khả năng chịu lỗi (fault tolerance), dữ liệu không chỉ được lưu trữ trên một node duy nhất. Thay vào đó, mỗi mẩu dữ liệu được sao lưu trên nhiều node khác nhau.

2.1.1. Sao lưu dựa trên vai trò Primary-Backup (Leader-Follower)

Tại cấp độ của từng phân mảnh dữ liệu, hệ thống hoạt động theo mô hình **Primary-Backup** (còn gọi là Leader-Follower):

1. **Node sở hữu (Primary):** Đối với một key nhất định, node được xác định bởi thuật toán hash partitioning đóng vai trò là **Primary**. Node này là điểm liên lạc duy nhất chịu trách nhiệm điều phối các thao tác ghi (write operations) cho key đó.
2. **Node bản sao (Backups/Replicas):** Mỗi node Primary sẽ có một hoặc nhiều node Backup. Trong thiết kế hiện tại, các node kế tiếp nó trong vòng ring hash (ví dụ: owner_id + 1, owner_id + 2) được chỉ định làm node Backup.

2.1.2. Sao lưu Bất đồng bộ (Asynchronous Replication)

Đây là quyết định thiết kế cốt lõi của hệ thống. Khi Primary nhận được một yêu cầu PUT:

1. Nó ghi dữ liệu vào bộ lưu trữ cục bộ của mình.
2. **Ngay lập tức**, nó gửi lại phản hồi thành công cho client (hoặc node đã chuyển tiếp yêu cầu).
3. **Song song và sau đó**, nó gửi các yêu cầu REPLICA_PUT đến các node Backup của mình. Primary **không chờ** các node Backup xác nhận đã ghi thành công.

Ưu điểm của phương pháp này:

- **Độ trễ ghi thấp (Low Write Latency):** Client nhận được phản hồi rất nhanh vì không phải chờ đợi quá trình sao lưu dữ liệu qua mạng đến nhiều node hoàn tất.
- **Tính sẵn sàng cao cho thao tác ghi:** Hệ thống vẫn có thể tiếp nhận yêu cầu ghi mới ngay cả khi một hoặc nhiều node Backup tạm thời không hoạt động hoặc bị chậm.

Nhược điểm:

- **Nguy cơ mất dữ liệu:** Nếu node Primary gặp sự cố ngay sau khi xác nhận với client nhưng trước khi gửi xong bản sao đến các node Backup, thao tác ghi đó có thể bị mất vĩnh viễn. Đây là sự đánh đổi kinh điển giữa hiệu năng và độ bền (durability).

2.2 Mô hình nhất quán: Tính nhất quán sau cùng

Hệ quả trực tiếp của việc sử dụng chiến lược sao lưu bất đồng bộ là hệ thống chỉ có thể đảm bảo **Tính nhất quán sau cùng**.

Định nghĩa:

Tính nhất quán sau cùng cam kết rằng, nếu không có thêm bất kỳ cập nhật nào cho một mẩu dữ liệu, thì *cuối cùng* tất cả các bản sao của mẩu dữ liệu đó trên mọi node sẽ hội tụ về cùng một giá trị cuối cùng.

Ý nghĩa trong thực tế:

- **Cửa sổ không nhất quán (Inconsistency Window):** Tồn tại một khoảng thời gian ngắn ngay sau khi một thao tác ghi (PUT) được xác nhận thành công. Trong "cửa sổ" này, một thao tác đọc (GET) cho cùng một key có thể trả về giá trị cũ nếu nó tình cờ được gửi đến một node Backup chưa kịp nhận được bản cập nhật từ Primary.
- **Đọc lại những gì bạn đã viết (Read-your-writes):** Hệ thống không đảm bảo tính nhất quán này. Một client vừa ghi thành công một giá trị có thể đọc lại ngay lập tức và nhận về giá trị cũ.

Tại sao lại chọn Eventual Consistency?

Trong nhiều ứng dụng web hiện đại (ví dụ: mạng xã hội, các hệ thống đề xuất, phân tích log), việc có thể đọc dữ liệu cũ trong một vài mili giây hoặc thậm chí vài giây là chấp nhận được. Đổi lại, hệ thống đạt được hai lợi ích cực kỳ quan trọng:

1. **Độ trễ thấp:** Mang lại trải nghiệm người dùng tốt hơn.
2. **Tính sẵn sàng cao:** Hệ thống vẫn hoạt động (cả đọc và ghi) ngay cả khi có sự cố mạng hoặc lỗi tạm thời ở một vài node, tránh được tình trạng "treo" toàn bộ hệ thống.

Đây là sự đánh đổi có chủ đích để ưu tiên **tính sẵn sàng (Availability)** và **khả năng chịu phân vùng mạng (Partition Tolerance)** hơn là **tính nhất quán tức thời (Immediate Consistency)**, phù hợp với định lý CAP.

3. Chiến lược xử lí node hỏng và khôi phục dữ liệu

Một hệ thống phân tán thực thụ phải có khả năng chống chọi với sự cố của từng thành phần mà không làm sụp đổ toàn bộ hệ thống. Phần này mô tả cơ chế phát hiện lỗi, chiến lược duy trì hoạt động khi có lỗi, và quy trình khôi phục dữ liệu cho một node sau khi nó hoạt động trở lại.

3.1 Phát hiện lỗi và giao thức heartbeat

Để các node biết được trạng thái của nhau, hệ thống sử dụng một giao thức **Heartbeat** (nhịp tim) đơn giản và hiệu quả.

- **Cơ chế:** Theo một chu kỳ cố định (ví dụ: mỗi 5 giây), mỗi node sẽ chủ động gửi một thông điệp PING đến tất cả các node khác trong cụm mà nó biết.
- **Phản hồi:** Khi một node nhận được thông điệp PING, nó sẽ ngay lập tức gửi lại một thông điệp PONG để xác nhận rằng nó vẫn đang hoạt động.
- **Đánh dấu lỗi:** Nếu một node (Node A) không nhận được phản hồi PONG từ một node khác (Node B) trong một khoảng thời gian chờ nhất định (timeout), Node A sẽ coi Node B là đã chết (dead) và cập nhật trạng thái này trong danh sách các node của mình. Thông tin này rất quan trọng cho các bước xử lý lỗi tiếp theo.

Cơ chế này đảm bảo rằng trạng thái của cụm được cập nhật liên tục và các node có thể phản ứng nhanh chóng khi một thành viên gặp sự cố.

3.2. Tính chịu lỗi (fault tolerance) để đảm bảo tính sẵn sàng

Khi một node đã bị đánh dấu là dead, hệ thống phải có chiến lược để các yêu cầu không bị thất bại.

3.2.1. Xử lý yêu cầu Đọc (GET)

Đây là kịch bản quan trọng nhất để duy trì tính sẵn sàng cho người dùng.

1. Client gửi yêu cầu GET một key đến một node bất kỳ (Node A).
2. Node A xác định được node sở hữu (Primary) của key này là Node B.
3. Node A phát hiện ra rằng Node B đã được đánh dấu là dead trong danh sách các node của nó.
4. Thay vì báo lỗi, Node A sẽ không cố gắng liên lạc với Node B. Nó sẽ thông minh chuyển hướng yêu cầu GET đến các node bản sao (Backups) của Node B (ví dụ: Node B+1, Node B+2).
5. Node bản sao đầu tiên còn sống nhận được yêu cầu sẽ đọc dữ liệu từ bộ lưu trữ cục bộ của mình và trả về cho Node A, sau đó Node A trả về cho client.

Nhờ có các bản sao và cơ chế chuyển hướng thông minh này, hệ thống vẫn có thể phục vụ các yêu cầu đọc ngay cả khi node chính chịu trách nhiệm cho dữ liệu đó đã bị lỗi.

3.2.2. Xử lý yêu cầu Ghi (PUT)

Quá trình ghi vẫn tiếp diễn một cách linh hoạt:

- Khi node Primary nhận một yêu cầu PUT, nó sẽ cố gắng gửi bản sao đến tất cả các node Backup đã được cấu hình.
- Nếu một trong các node Backup đã chết, Primary chỉ đơn giản là bỏ qua việc gửi bản sao đến node đó và tiếp tục gửi cho các node còn lại.
- Thao tác ghi vẫn được coi là thành công và client nhận được phản hồi nhanh chóng. Dữ liệu sẽ tạm thời bị mất đi một bản sao, nhưng tính nhất quán sẽ được phục hồi khi node bị lỗi hoạt động trở lại.

3.3 Quy trình khôi phục dữ liệu (Node recovery)

Khi một node bị lỗi khởi động lại và tham gia vào cụm, nó cần phải cập nhật lại dữ liệu mà nó đã bỏ lỡ trong thời gian "offline".

1. **Thông báo và Yêu cầu:** Node vừa khởi động sẽ gửi một thông điệp đặc biệt, NODE_RECOVERY, đến một node bất kỳ còn sống trong cụm. Thông điệp này chứa id của chính nó.
2. **Tổng hợp dữ liệu:** Node nhận được yêu cầu NODE_RECOVERY sẽ duyệt qua toàn bộ kho dữ liệu (self.data) của mình. Nó sẽ tìm và tập hợp tất cả các cặp key-value mà node đang cần khôi phục (id trong yêu cầu) phải chịu trách nhiệm (với vai trò là Primary hoặc Backup).
3. **Gửi gói khôi phục:** Tất cả dữ liệu đã được tổng hợp sẽ được đóng gói và gửi lại cho node mới khởi động.
4. **Cập nhật dữ liệu:** Node mới khởi động nhận được gói dữ liệu và ghi toàn bộ vào bộ lưu trữ cục bộ của mình.

Sau khi quá trình này hoàn tất, node đã được khôi phục hoàn toàn, chứa đầy đủ dữ liệu nó cần và sẵn sàng tham gia xử lý các yêu cầu mới, đồng thời khôi phục lại mức độ sao lưu ban đầu của hệ thống. Quá trình này được tóm tắt qua chu trình: **Replication ➔ Failure ➔ Fault Tolerance ➔ Recovery**.

4. Phương án phân chia và mở rộng

Phần này mô tả phương pháp được sử dụng để phân tán dữ liệu trên toàn cụm và thảo luận về chiến lược mở rộng hệ thống để xử lý tải trọng và lượng dữ liệu ngày càng tăng.

4.1 Phân chia dữ liệu: hash Partitioning

Để phân phối các cặp key-value một cách đồng đều trên tất cả các node, hệ thống sử dụng một kỹ thuật phổ biến và đơn giản là **Hash Partitioning** (Phân chia dựa trên hàm băm).

- **Thuật toán:**
    1. Khi một key cần được định vị (để đọc, ghi hoặc xóa), hệ thống sẽ áp dụng một hàm băm mật mã học (cụ thể là SHA-256) lên key đó để tạo ra một chuỗi giá trị băm.
    2. Giá trị băm này sau đó được chuyển thành một số nguyên.
    3. Cuối cùng, hệ thống lấy kết quả số nguyên đó thực hiện phép toán **modulo (%)** với tổng số node hiện có trong cụm (N). Kết quả của phép toán này chính là id của node sở hữu (owner) key đó.
- owner_id = hash(key) % N
- **Ưu điểm:**
    - **Đơn giản:** Thuật toán rất dễ thực hiện và dễ hiểu.
    - **Phân phối tương đối đồng đều:** Với một hàm băm tốt như SHA-256, các key sẽ được phân bố ngẫu nhiên và đồng đều trên các node, giúp cân bằng tải và dung lượng lưu trữ, tránh tình trạng một node bị quá tải trong khi các node khác lại nhàn rỗi.
- **Nhược điểm (Hạn chế lớn khi mở rộng):**
    - **Vấn đề "Rehashing toàn bộ":** Nhược điểm nghiêm trọng của phương pháp này xuất hiện khi số lượng node (N) thay đổi.
        - Khi một node mới được thêm vào (N → N+1), công thức hash(key) % N sẽ thay đổi hoàn toàn. Kết quả là, gần như **tất cả các key** trong hệ thống sẽ được ánh xạ đến một node sở hữu mới.
        - Điều này đòi hỏi một cuộc di chuyển dữ liệu khổng lồ và tốn kém trên toàn bộ hệ thống, gây ra tình trạng quá tải mạng và có thể làm giảm hiệu năng hệ thống một cách đáng kể trong quá trình tái phân bổ.
    - Tương tự, khi một node bị xóa hoặc bị lỗi vĩnh viễn (N → N-1), một cuộc tái phân bổ dữ liệu lớn cũng xảy ra.

4.2 Chiến lược mở rộng và đề xuất cải tiến

Hệ thống được thiết kế để **mở rộng theo chiều ngang (horizontal scaling)**, tức là tăng cường năng lực bằng cách thêm nhiều máy chủ (node) hơn, thay vì nâng cấp phần cứng của các máy chủ hiện có (vertical scaling). Tuy nhiên, để việc mở rộng thực sự hiệu quả và giảm thiểu chi phí tái phân bổ dữ liệu, cần một phương pháp phân chia thông minh hơn.

Đề xuất cải tiến: Sử dụng Consistent Hashing (Băm nhất quán)

Consistent Hashing là một kỹ thuật được thiết kế đặc biệt để giải quyết vấn đề "rehashing toàn bộ" của phương pháp modulo.

- **Cách hoạt động:**
    1. Thay vì một không gian số tuyến tính, hãy tưởng tượng một **vòng tròn (ring)** đại diện cho toàn bộ không gian của các giá trị băm (ví dụ: từ 0 đến 2^160 - 1).
    2. Mỗi node trong cụm được băm và đặt vào một vị trí trên vòng tròn này.
    3. Mỗi key cũng được băm và đặt vào một vị trí trên vòng tròn.
    4. Để tìm node sở hữu một key, hệ thống sẽ đi theo chiều kim đồng hồ từ vị trí của key trên vòng tròn cho đến khi gặp node đầu tiên. Node đó chính là chủ sở hữu.
- **Lợi ích khi mở rộng:**
    - **Thêm node:** Khi một node mới được thêm vào và đặt trên vòng tròn, nó chỉ chịu trách nhiệm cho các key nằm giữa nó và node liền trước nó theo chiều kim đồng hồ. Điều này có nghĩa là chỉ có một phần nhỏ dữ liệu từ *một node duy nhất* (node liền sau nó) cần được di chuyển sang node mới.
    - **Xóa node:** Khi một node bị xóa khỏi vòng tròn, toàn bộ dữ liệu của nó sẽ được chuyển giao cho node liền sau nó trên vòng tròn.
    - Trong cả hai trường hợp, sự thay đổi chỉ ảnh hưởng cục bộ đến các node lân cận trên vòng tròn, và đại đa số dữ liệu trên các node khác không bị ảnh hưởng.

Việc chuyển đổi từ Hash Partitioning sang Consistent Hashing là bước cải tiến quan trọng nhất để hệ thống có thể mở rộng một cách mượt mà và hiệu quả từ vài node lên đến hàng trăm hoặc hàng ngàn node.

5. Những hạn chế và đề xuất cải tiến

Mặc dù hệ thống hiện tại đã triển khai được các chức năng cốt lõi của một kho lưu trữ key-value phân tán, nó vẫn còn tồn tại một số hạn chế quan trọng. Phần này sẽ phân tích các hạn chế đó và đề xuất những hướng cải tiến cụ thể để xây dựng một hệ thống mạnh mẽ và hoàn thiện hơn.

5.1. Hạn chế về mở rộng: chi phí tái phân bố dữ liệu cao

- **Vấn đề:** Như đã phân tích ở phần 4, việc sử dụng thuật toán **Hash Partitioning (modulo)** là điểm yếu lớn nhất của hệ thống khi cần mở rộng hoặc thu hẹp. Bất kỳ sự thay đổi nào về số lượng node đều buộc gần như toàn bộ dữ liệu phải được di chuyển, gây tốn kém tài nguyên mạng và CPU, có thể làm hệ thống ngừng trệ trong quá trình thay đổi cấu trúc.
- **Đề xuất cải tiến:**
    - **Áp dụng Băm nhất quán (Consistent Hashing):** Đây là ưu tiên cải tiến hàng đầu. Chuyển đổi sang Consistent Hashing sẽ giảm thiểu đáng kể lượng dữ liệu cần di chuyển khi thêm/bớt node, làm cho quá trình mở rộng trở nên mượt mà và hiệu quả hơn rất nhiều.

5.2. Hạn chế về độ bền dữ liệu (Durability)

- **Vấn đề:**
    1. **Mất dữ liệu khi ghi:** Do sử dụng sao lưu bất đồng bộ, có một rủi ro nhỏ nhưng hiện hữu: nếu node Primary gặp sự cố ngay sau khi phản hồi cho client nhưng trước khi kịp gửi bản sao đi, thao tác ghi đó sẽ bị mất vĩnh viễn.
    2. **Thiếu bộ nhớ bền vững:** Toàn bộ dữ liệu hiện đang được lưu trữ trong bộ nhớ RAM của các node. Nếu tất cả các node trong cụm cùng lúc bị mất điện hoặc khởi động lại, toàn bộ dữ liệu sẽ bị xóa sạch.
- **Đề xuất cải tiến:**
    - **Cơ chế ghi đồng bộ tùy chọn (Optional Synchronous Writes):** Cung cấp một tùy chọn cho client để yêu cầu một thao tác ghi an toàn hơn. Khi đó, node Primary sẽ chỉ phản hồi thành công sau khi đã nhận được xác nhận từ một số lượng (W) các node bản sao. Điều này tăng độ trễ ghi nhưng đảm bảo dữ liệu không bị mất.
    - **Tích hợp bộ lưu trữ bền vững (Persistent Storage):** Mỗi node cần ghi dữ liệu của nó xuống đĩa. Có thể sử dụng các giải pháp từ đơn giản đến phức tạp:
        - Ghi vào một file log (Write-Ahead Log - WAL).

        - Tích hợp với một thư viện lưu trữ key-value hiệu năng cao như RocksDB hoặc LevelDB.

5.3. Hạn chế về tính sẵn sàng của thao tác ghi

- **Vấn đề:** Khi một node Primary bị lỗi, hệ thống có thể tiếp tục phục vụ yêu cầu đọc từ các bản sao. Tuy nhiên, không có cơ chế tự động nào để **thúc đẩy một trong các node bản sao lên làm Primary mới**. Do đó, tất cả các key thuộc về node đã chết sẽ không thể thực hiện thao tác ghi (PUT) cho đến khi node đó được khôi phục.
- **Đề xuất cải tiến:**
    - **Triển khai Thuật toán Bầu cử Leader (Leader Election):** Trong nhóm các node sao lưu cho một phân mảnh, cần triển khai một thuật toán đồng thuận (ví dụ: một phiên bản đơn giản của Raft hoặc Paxos). Khi Primary bị phát hiện là đã chết, các node bản sao còn lại sẽ tự động "bầu cử" ra một Primary mới để tiếp quản các thao tác ghi, đảm bảo tính sẵn sàng cho cả việc ghi dữ liệu.

5.4. Hạn chế trong quy trình khôi phục

- **Vấn đề:** Quy trình khôi phục hiện tại dựa vào việc một node vừa khởi động yêu cầu dữ liệu từ một node khác. Điều này có thể tạo ra gánh nặng lớn cho node "giúp đỡ" và có thể không đáng tin cậy nếu chính node đó cũng có dữ liệu chưa nhất quán.
- **Đề xuất cải tiến:**
    - **Sử dụng Cây Merkle (Merkle Trees) để chống Entropy:** Để khôi phục hiệu quả, các node nên định kỳ so sánh dữ liệu của mình với các bản sao khác. Cây Merkle cho phép các node nhanh chóng phát hiện sự khác biệt trong tập dữ liệu của chúng mà không cần gửi toàn bộ dữ liệu qua mạng. Chúng chỉ cần đồng bộ hóa những phần khác biệt, giúp quá trình tự chữa lành (self-healing) diễn ra liên tục và hiệu quả.